

Logique et satisfaisabilité : présentation du fascicule

Le fascicule précédent décrivait un problème par une liste d'états et d'actions. Celui-ci adopte l'approche inverse : on décrit ce qui doit être vrai, et l'on demande à une procédure automatique de trouver une situation qui respecte cette description.

Cette façon de procéder a deux avantages. Elle sépare la description du problème de sa résolution, et elle permet de décrire des problèmes dont on ne sait pas énumérer les états à l'avance.

Le prix à payer est une difficulté théorique : décider si une formule logique admet une solution est un problème complet pour la classe NP. Aucun algorithme connu ne le résout en temps polynomial dans le pire cas.

En pratique, cela n'empêche pas de traiter des instances de plusieurs millions de variables. La différence entre la théorie du pire cas et la pratique industrielle est l'un des enseignements de ce fascicule.

Pourquoi formaliser en logique

Un langage formel élimine l'ambiguïté. Une phrase en langue naturelle peut recevoir plusieurs interprétations ; une formule logique en reçoit une seule, fixée par les règles du langage.

Un langage formel permet le raisonnement automatique. Une fois le problème traduit, une procédure générale peut chercher une solution sans connaître la signification des symboles.

Un langage formel rend les contraintes vérifiables. On peut demander pourquoi une solution est refusée, en exhibant la contrainte violée ou la clause insatisfaite.

Ces trois propriétés expliquent l'usage de la logique dans la vérification de programmes, la configuration de systèmes et la planification.

Syntaxe de la logique propositionnelle

Le vocabulaire de base est constitué de variables propositionnelles, notées habituellement p , q , r . Chacune représente une affirmation qui peut être vraie ou fausse.

Les connecteurs sont la négation, la conjonction, la disjonction, l'implication et l'équivalence. Ils permettent de construire des formules à partir des variables.

Une formule bien formée est définie récursivement : une variable est une formule, et l'application d'un connecteur à des formules produit une formule.

Les parenthèses lèvent l'ambiguïté de lecture. Par convention, la négation lie plus fort que la conjonction, qui lie plus fort que la disjonction.

Sémantique et tables de vérité

Une affectation associe à chaque variable une valeur de vérité. Une formule est évaluée sous une affectation donnée en appliquant les règles de chaque connecteur.

La table de vérité énumère les valeurs de la formule pour toutes les affectations possibles. Avec n variables, elle comporte deux puissance n lignes, ce qui interdit son usage au-delà de quelques dizaines de variables.

Une formule est satisfaisable s'il existe au moins une affectation qui la rend vraie. Elle est valide si toutes les affectations la rendent vraie, et insatisfaisable dans le cas contraire.

Une formule est insatisfaisable si et seulement si sa négation est valide. Ce lien entre les deux notions est utilisé par de nombreuses procédures de décision.

Équivalences utiles

Lois de De Morgan. La négation d'une conjonction est la disjonction des négations, et réciproquement. Ces lois permettent de faire descendre les négations jusqu'aux variables.

Distributivité. La conjonction se distribue sur la disjonction, et la disjonction sur la conjonction. C'est la distributivité qui produit la forme normale conjonctive.

Implication et équivalence. Une implication est équivalente à la disjonction de sa prémisse niée et de sa conclusion. Cette réécriture supprime un connecteur lors de la conversion en forme normale.

Ces équivalences sont utilisées mécaniquement par les procédures de conversion. Elles ne changent pas la signification de la formule, seulement son écriture.

Raisonnement et règles d'inférence

Une règle d'inférence produit de nouvelles formules à partir de formules connues. Le modus ponens, qui déduit la conclusion d'une implication et de sa prémisse, en est l'exemple le plus connu.

Le raisonnement par résolution utilise une seule règle. À partir de deux clauses contenant un littéral et son opposé, on produit la clause formée des autres littéraux.

Cette règle est complète pour la réfutation : si un ensemble de clauses est insatisfaisable, la résolution peut en dériver la clause vide. Chercher une solution revient donc à tenter de dériver cette clause vide.

Le raisonnement automatique se ramène ainsi à une question de recherche : quelles résolutions appliquer, dans quel ordre, pour parvenir à la clause vide sans exploser en combinatoire.

Un exemple complet de formalisation

Soit un système qui doit respecter trois contraintes : si la sauvegarde automatique est activée, les journaux sont archivés ; les journaux ne sont pas archivés si le disque est plein ; le disque est plein ou la sauvegarde automatique est activée.

Ces trois contraintes s'écrivent comme trois formules. Le système est cohérent si ces formules peuvent être rendues vraies simultanément.

Si l'on ajoute une quatrième contrainte affirmant que le disque n'est pas plein, la conjonction reste satisfaisable, mais elle impose la valeur des autres variables. Si l'on ajoute au contraire que les journaux ne sont pas archivés, les contraintes deviennent contradictoires.

Cet exemple illustre l'usage réel de la logique propositionnelle : vérifier qu'un ensemble de règles ne se contredit pas, et déterminer quelles configurations les satisfont.

Le problème SAT

Le problème SAT prend en entrée une formule propositionnelle et demande si elle est satisfaisable. En cas de réponse positive, on demande souvent une affectation témoin.

La forme la plus étudiée est SAT sous forme normale conjonctive, c'est-à-dire une conjonction de clauses. Toute formule peut y être ramenée.

Les cas particuliers sont plus faciles. Si chaque clause comporte au plus un littéral, le problème est résolu par propagation. Si chaque clause en comporte au plus deux, le problème est résolu par une procédure fondée sur les composantes fortement connexes.

La difficulté apparaît dès que certaines clauses comportent trois littéraux. Cette variante, appelée 3-SAT, est complète pour la classe NP.

SAT et la classe NP

En 1971, Stephen Cook a démontré que SAT est complet pour la classe NP, c'est-à-dire qu'il appartient à NP et que tout problème de NP s'y ramène en temps polynomial. Leonid Levin a obtenu un résultat analogue de façon indépendante.

Ce résultat a une conséquence pratique : si l'on trouvait un algorithme polynomial pour SAT, on en déduirait des algorithmes polynomiaux pour des milliers d'autres problèmes, de la planification à la vérification de circuits.

Aucun algorithme de ce type n'est connu, et l'hypothèse contraire reste ouverte. En revanche, aucun résultat ne prouve non plus qu'il n'en existe pas.

Le caractère NP-complet ne signifie pas que le problème est insoluble, mais que l'on ne peut pas espérer d'algorithme à la fois général et rapide dans le pire cas.

La forme normale conjonctive

Une clause est une disjonction de littéraux, un littéral étant une variable ou sa négation. Une formule est en forme normale conjonctive, abrégée CNF, lorsqu'elle est une conjonction de clauses.

La forme normale conjonctive est l'entrée standard des solveurs. Elle a deux avantages : la satisfaction se vérifie clause par clause, et la plupart des techniques de propagation s'expriment directement sur les clauses.

Un ensemble de clauses est satisfait si au moins un littéral est vrai dans chaque clause. La clause vide, qui ne contient aucun littéral, est donc insatisfaisable par convention.

Deux ensembles de clauses équivalents ont les mêmes modèles, mais leur taille peut différer considérablement selon la méthode de conversion employée.

Convertir une formule en CNF

La méthode directe applique les équivalences : on élimine les implications et les équivalences, on fait descendre les négations, puis on distribue la disjonction sur la conjonction.

Cette méthode préserve l'équivalence logique, mais peut faire croître la taille de la formule de façon exponentielle. Une formule de la forme conjonction de disjonctions en est l'exemple classique.

La transformation de Tseitin évite cette croissance en introduisant une variable auxiliaire pour chaque sous-formule. La formule obtenue n'est plus équivalente, mais elle est équisatisfaisable : elle admet un modèle si et seulement si l'originale en admet un.

La taille obtenue est linéaire en la taille de la formule d'origine. Tous les solveurs pratiques utilisent cette transformation, ou une variante.

L'algorithme DPLL

L'algorithme DPLL, publié dans les années 1960 par Davis, Putnam, Logemann et Loveland, reste la base des solveurs modernes. Il combine deux idées : la propagation et le choix.

La propagation unitaire traite les clauses où il ne reste qu'un littéral non assigné. Ce littéral doit être vrai, sinon la clause serait fausse. On l'assigne donc et l'on propage à nouveau.

Lorsque plus aucune clause unitaire n'existe, l'algorithme choisit une variable non assignée et explore successivement ses deux valeurs. En cas d'échec, il revient sur ce choix.

L'algorithme est complet : il explore systématiquement l'espace des affectations, mais la propagation élimine de larges portions de cet espace avant toute exploration.

Heuristiques de choix de variable

Le choix de la variable à assigner détermine l'efficacité pratique du solveur. Une mauvaise politique peut multiplier le nombre de noeuds explorés par plusieurs ordres de grandeur.

L'heuristique la plus simple choisit la variable la plus fréquente dans les clauses courtes non satisfaites. Cette politique privilégie les variables qui contraignent le plus le problème.

Depuis les années 1990, les solveurs utilisent des scores de littéraux décroissants au fil du temps. Ces scores sont réinitialisés régulièrement depuis les valeurs mesurées dans les conflits récents.

Une fois la variable choisie, la valeur à essayer en premier est souvent celle qui satisfait le plus grand nombre de clauses. Ce choix ne change pas le pire cas, mais accélère la découverte d'un modèle dans les cas satisfaisables.

Apprentissage et retours arrière modernes

Le point faible de DPLL est la répétition des erreurs : lorsqu'un conflit survient, il n'en garde aucune trace et peut retomber sur la même configuration quelques branches plus loin.

Les solveurs dits CDCL, pour conflit dirigé et apprentissage de clauses, analysent le conflit pour déterminer la partie de l'affectation qui l'a causé. Ils en déduisent une nouvelle clause, appelée clause apprise, ajoutée à la formule.

Cette clause interdit de reproduire la même combinaison de valeurs. Elle accélère la recherche en éliminant définitivement certaines régions de l'espace de recherche.

Par ailleurs, le retour arrière n'est plus chronologique mais dirigé : on remonte directement au point qui a causé le conflit, au lieu de dépiler un choix après l'autre. Ces deux techniques expliquent l'écart entre les solveurs des années 1990 et ceux d'aujourd'hui.

Ce que les solveurs savent faire aujourd'hui

Les solveurs modernes traitent couramment des instances industrielles de plusieurs centaines de milliers de variables et de millions de clauses.

Leur efficacité vient de la combinaison de plusieurs techniques : propagation unitaire optimisée par des structures de données adaptées, apprentissage de clauses, redémarrages périodiques et heuristiques dynamiques.

Des compétitions annuelles comparent les solveurs sur des familles d'instances variées. Les résultats montrent que les progrès sont continus, d'environ un ordre de grandeur tous les cinq ans.

Cette progression a des conséquences pratiques : de nombreux problèmes de configuration, d'ordonnancement ou de vérification formelle se ramènent à SAT et sont désormais résolus efficacement.

Problèmes de satisfaction de contraintes

Un problème de satisfaction de contraintes, abrégé CSP, est défini par un ensemble de variables, un domaine de valeurs pour chaque variable et un ensemble de contraintes portant sur des combinaisons de variables.

Une affectation est complète si toutes les variables reçoivent une valeur. Une contrainte est satisfaite si la combinaison de valeurs qu'elle porte vérifie la relation attendue.

Une solution est une affectation complète qui satisfait toutes les contraintes. Le problème peut être purement décisionnel, ou demander l'optimisation d'un critère parmi les solutions.

La différence avec SAT tient à la forme du domaine : les variables d'un CSP prennent leurs valeurs dans un ensemble quelconque, pas nécessairement booléen.

Exemples classiques de CSP

Coloration de graphes. On veut assigner une couleur à chaque sommet de telle sorte que deux sommets reliés n'aient jamais la même. Le domaine est l'ensemble des couleurs disponibles.

Ordonnancement de tâches. Chaque tâche a une durée, une fenêtre de temps possible et des ressources. Les contraintes expriment que certaines tâches ne peuvent se chevaucher.

Puzzle logique. Le placement de nombres dans une grille de type sudoku, ou celui de dames sur un échiquier sans mises en prise, sont les exemples les plus courants.

Ces trois familles partagent la même structure : un grand nombre de contraintes locales, dont la combinaison rend la recherche globale difficile.

Résolution par retour sur trace

L'algorithme de base parcourt les variables dans un ordre fixé. Il assigne une valeur à la première variable, puis passe à la suivante, en vérifiant après chaque assignation que les contraintes restent satisfaisables.

Si aucune valeur ne convient, l'algorithme revient à la variable précédente et essaie une autre valeur. C'est un parcours systématique de l'arbre des affectations complètes.

Sa complexité est celle du nombre d'affectations, mais une vérification anticipée des contraintes permet d'élaguer de larges sous-arbres avant de les explorer.

Vérifier seulement les contraintes portant sur la nouvelle variable est plus efficace que de revérifier la totalité de l'affectation à chaque étape.

Propagation de contraintes

La propagation consiste à propager les conséquences d'une assignation : si une valeur devient impossible pour une variable, on retire cette valeur de son domaine.

La vérification vers l'avant filtre les domaines des variables non assignées à chaque étape. Le filtrage est simple, mais il ne détecte pas les contradictions qui ne concernent que des variables non assignées.

La cohérence d'arc, obtenue par un algorithme classique, est plus coûteuse mais beaucoup plus informative. Deux variables sont cohérentes d'arc si toute valeur du domaine de l'une possède un support dans le domaine de l'autre.

Faire converger ces cohérences locales détecte l'échec bien plus tôt qu'une simple vérification vers l'avant, au prix d'un coût de calcul supérieur à chaque étape.

Heuristiques de choix dans un CSP

Choisir la prochaine variable. L'heuristique dite du minimum de valeurs restantes sélectionne la variable dont le domaine est le plus petit, car c'est la plus contrainte.

En cas d'égalité, on choisit la variable impliquée dans le plus grand nombre de contraintes avec les variables non assignées. Cette seconde heuristique favorise les variables centrales du problème.

Choisir la prochaine valeur. On essaie en premier la valeur qui contraint le moins les autres variables, afin de laisser le maximum de liberté à la suite de la recherche.

Choisir le point de retour. Lorsqu'un échec survient, revenir sur la variable qui a causé le conflit plutôt que sur la dernière assignée évite de refaire systématiquement les mêmes explorations.

Recherche locale sur les CSP

La méthode dite des conflits minimaux commence par une affectation complète, éventuellement mauvaise, puis répare les contraintes violées.

À chaque étape, elle choisit une variable impliquée dans un conflit et lui assigne la valeur qui minimise le nombre de conflits. La procédure s'arrête lorsqu'aucun conflit ne subsiste.

Cet algorithme surprend par son efficacité sur de grandes instances, comme le placement de plusieurs milliers de reines sur un échiquier, alors que la recherche systématique y échoue.

Il ne fournit aucune garantie : il peut s'arrêter sur un état sans conflit, ou tourner indéfiniment si l'instance est insatisfaisable. En pratique, on lui impose un budget de tentative et on le redémarre avec une affectation initiale différente.

SAT et CSP : deux vues du même problème

Les deux formalismes sont équivalents du point de vue de la difficulté. On encode un CSP en SAT en associant une variable booléenne à chaque couple variable-valeur, puis en écrivant les contraintes comme des clauses.

Réciproquement, une formule SAT se voit comme un CSP dont les variables sont booléennes et dont chaque clause est une contrainte interdisant une combinaison précise.

Le choix du formalisme est donc une question d'ergonomie et d'efficacité pratique. L'encodage en SAT bénéficie de solveurs très optimisés ; le modèle CSP conserve une structure plus lisible, utile pour expliquer les solutions.

En pratique, de nombreux outils traduisent un CSP en SAT, résolvent la formule, puis reconvertissent le modèle obtenu en affectation des variables d'origine.

Limites et pièges

Un encodage maladroit peut rendre un problème facile artificiellement difficile. Ajouter des variables auxiliaires inutiles ou écrire une contrainte de façon redondante dégrade les performances de plusieurs ordres de grandeur.

La taille de l'encodage compte autant que sa correction. Une traduction linéaire mais peu contrainte se propage mal ; une traduction très contrainte mais quadratique peut saturer la mémoire.

Enfin, la symétrie est un piège classique. Un problème dont les solutions sont équivalentes par permutation oblige le solveur à explorer chacune d'elles, sauf si l'on ajoute des contraintes qui brisent ces symétries.

Ces questions d'encodage constituent l'essentiel du travail sur les problèmes réels : le solveur, lui, est souvent utilisé tel quel.

Ce qu'il faut retenir de ce fascicule

La logique propositionnelle fournit un langage sans ambiguïté pour décrire des contraintes. Toute formule peut être ramenée en forme normale conjonctive, qui est l'entrée standard des solveurs.

SAT est complet pour NP, mais cela ne l'empêche pas d'être résolu efficacement sur des instances industrielles. La différence tient à la propagation, à l'apprentissage de clauses et aux heuristiques dynamiques.

Les problèmes de satisfaction de contraintes offrent une formulation plus naturelle lorsque les domaines ne sont pas booléens. Les techniques de propagation y jouent le même rôle que la propagation unitaire dans SAT.

Enfin, la qualité de l'encodage détermine souvent plus les performances que le choix du solveur. C'est la partie du travail qui demande le plus de soin.

Exercices du fascicule 2

Exercice 1. Mettre la formule suivante en forme normale conjonctive, d'abord par équivalences, puis par la transformation de Tseitin. Comparer la taille des deux résultats.

Exercice 2. Appliquer DPLL à la main sur un ensemble de cinq clauses à quatre variables. Indiquer à chaque étape les clauses devenues unitaires et les affectations propagées.

Exercice 3. Encoder le problème de coloration d'un graphe à cinq sommets avec trois couleurs en SAT. Indiquer le nombre de variables et de clauses obtenues.

Exercice 4. Résoudre une instance de quatre reines par la méthode des conflits minimaux, en partant d'une affectation initiale choisie arbitrairement, puis par retour sur trace avec propagation.